

DAY5_WEEK_4_HANDSON_CSHARP_Gurunath_Rageni

Level-1 Problem 1: Safe Division Calculator

Scenario:

A calculator application must perform division operations safely. If the user enters an invalid value or tries to divide by zero, the system should handle the error gracefully instead of crashing.

Requirements:

1. Create a class **Calculator**.
2. Create a method **Divide(int numerator, int denominator)**.
3. Use **try-catch-finally** blocks to handle errors.
4. If the denominator is zero, display an appropriate error message.
5. Ensure the program continues execution after handling the error.

Technical Constraints:

1. Use **try-catch-finally** blocks for exception handling.
2. Handle **DivideByZeroException**.
3. Ensure the **finally block always executes**.

Expectations:

- Demonstrate proper use of try-catch-finally blocks.
- Display meaningful error messages.
- Ensure program stability even when errors occur.

Learning Outcome:

- Understand basic exception handling.
- Use try-catch-finally blocks effectively.
- Handle runtime errors safely.

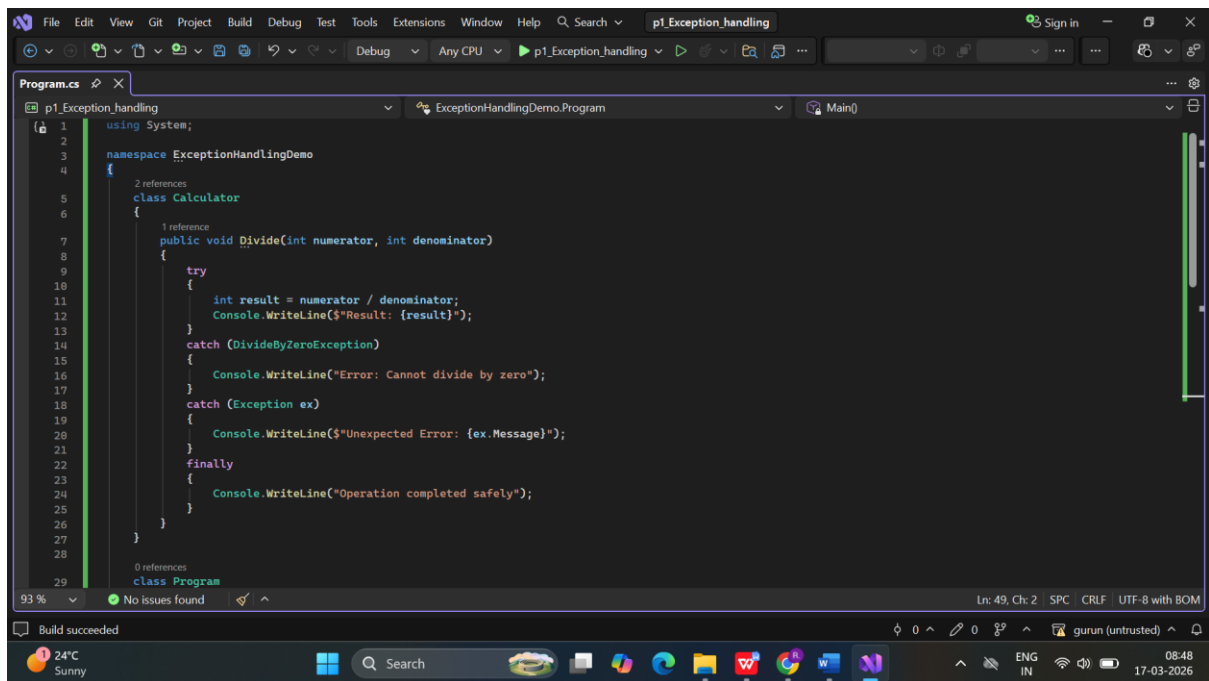
Sample Input:

Numerator = 20
Denominator = 0

Sample Output:

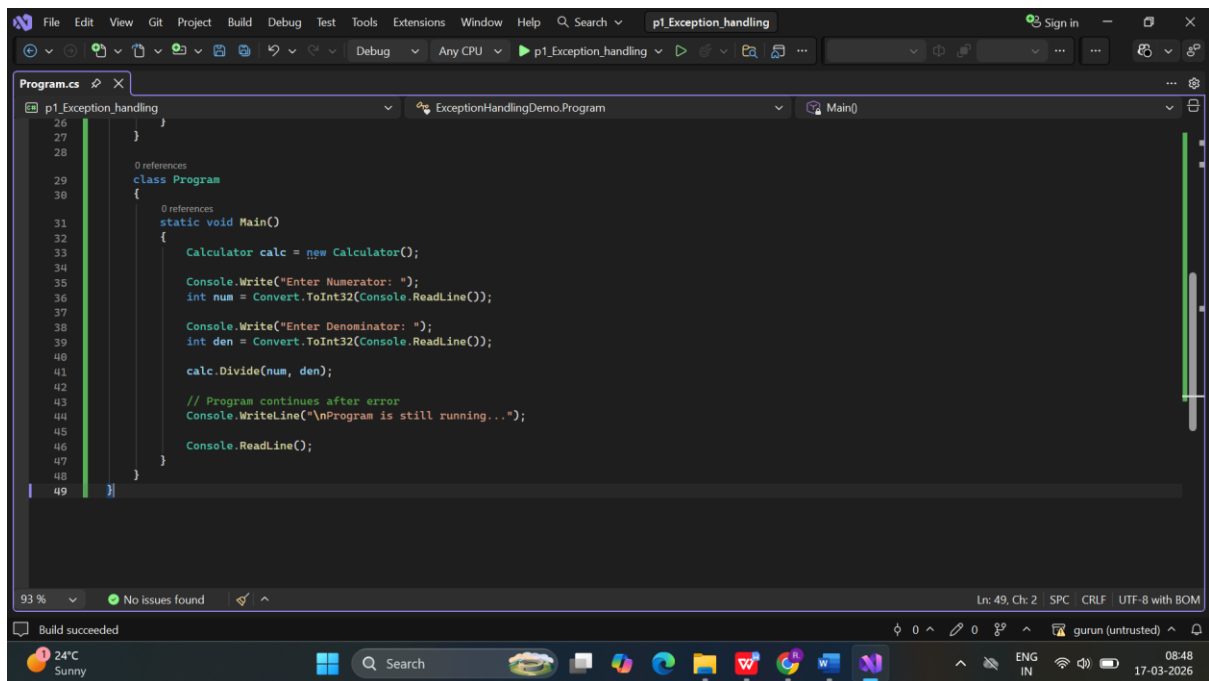
Error: Cannot divide by zero
Operation completed safely

Code Screenshot:



This screenshot shows the Visual Studio IDE with a C# project named 'p1_Exception_handling'. The 'Program.cs' file is open, displaying the 'ExceptionHandlingDemo' namespace and a 'Calculator' class. The 'Calculator' class has a 'Divide' method that takes two integers, 'numerator' and 'denominator', and returns an integer 'result'. The method uses a try-catch-finally block to handle exceptions. The 'try' block calculates the result and prints it. The 'catch' block catches a 'DivideByZeroException' and prints an error message. The 'finally' block prints a message indicating the operation completed safely. The 'Program' class has a 'Main' method that calls 'Calculator.Divide'.

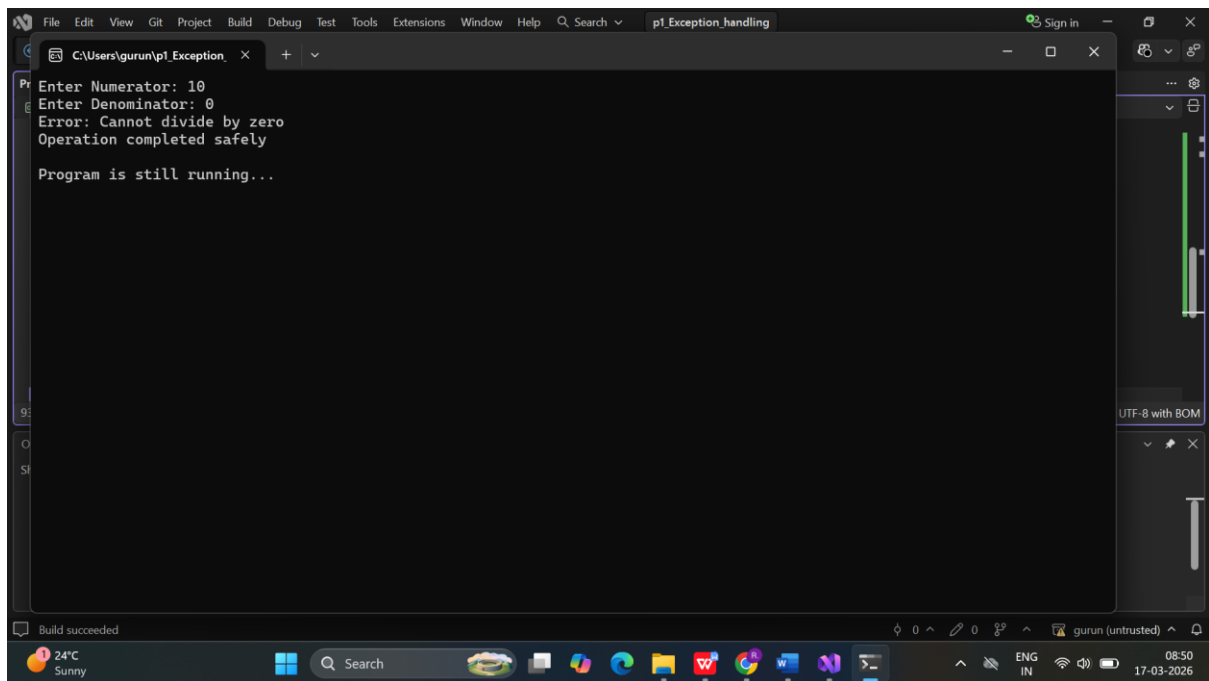
```
1 using System;
2
3 namespace ExceptionHandlingDemo
4 {
5     2 references
6     class Calculator
7     {
8         1 reference
9         public void Divide(int numerator, int denominator)
10        {
11            try
12            {
13                int result = numerator / denominator;
14                Console.WriteLine($"Result: {result}");
15            }
16            catch (DivideByZeroException)
17            {
18                Console.WriteLine("Error: Cannot divide by zero");
19            }
20            catch (Exception ex)
21            {
22                Console.WriteLine($"Unexpected Error: {ex.Message}");
23            }
24            finally
25            {
26                Console.WriteLine("Operation completed safely");
27            }
28        }
29    }
30
31    0 references
32    class Program
33    {
34    }
```



This screenshot shows the Visual Studio IDE with the same C# project. The 'Program.cs' file is open, displaying the 'Program' class. The 'Program' class has a 'Main' method that creates a new 'Calculator' object, prompts the user for a numerator and denominator, and calls the 'Divide' method. The program continues to run after the exception is handled, printing a message and waiting for user input.

```
26 }
27
28
29 0 references
30 class Program
31 {
32     0 references
33     static void Main()
34     {
35         Calculator calc = new Calculator();
36
37         Console.Write("Enter Numerator: ");
38         int num = Convert.ToInt32(Console.ReadLine());
39
40         Console.Write("Enter Denominator: ");
41         int den = Convert.ToInt32(Console.ReadLine());
42
43         calc.Divide(num, den);
44
45         // Program continues after error
46         Console.WriteLine("\nProgram is still running...");
47
48         Console.ReadLine();
49     }
50 }
```

Output Screenshot:

A screenshot of a Windows terminal window titled 'p1_Exception_handling'. The terminal shows the following output: 'Enter Numerator: 10', 'Enter Denominator: 0', 'Error: Cannot divide by zero', 'Operation completed safely', and 'Program is still running...'. The window has a dark theme and a sidebar on the right with a file explorer and a search bar. The Windows taskbar is visible at the bottom with various icons and the system clock showing 08:50 on 17-03-2026.

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search p1_Exception_handling
C:\Users\gurun\p1_Exception_handling
Pr
Enter Numerator: 10
Enter Denominator: 0
Error: Cannot divide by zero
Operation completed safely
Program is still running...
9:
O
S
Build succeeded
24°C Sunny
Search
ENG IN
08:50 17-03-2026
```

Level-2 Problem 1: Bank Withdrawal System with Custom Exception

Scenario:

A bank wants to implement a withdrawal system that throws a custom exception when a customer attempts to withdraw more money than available in their account.

Requirements:

1. Create a class **BankAccount** with a private field **balance**.
2. Create a method **Withdraw(double amount)**.
3. Create a **custom exception class InsufficientBalanceException**.
4. Throw the custom exception if the withdrawal amount exceeds the balance.
5. Handle the exception in the main program using **try-catch**.
6. Ensure proper cleanup using a **finally block**.

Technical Constraints:

- Use **custom exception classes** by inheriting from Exception.
- Implement **error propagation** by throwing exceptions from methods.
- Handle exceptions in the calling method.

Expectations:

- Demonstrate creation and usage of custom exceptions.

- Implement proper exception handling in calling code.
- Display user-friendly error messages.

Learning Outcome:

- Learn how to create **custom exceptions**.
- Understand **throw and error propagation**.
- Apply structured exception handling in real-world scenarios.

Sample Input:

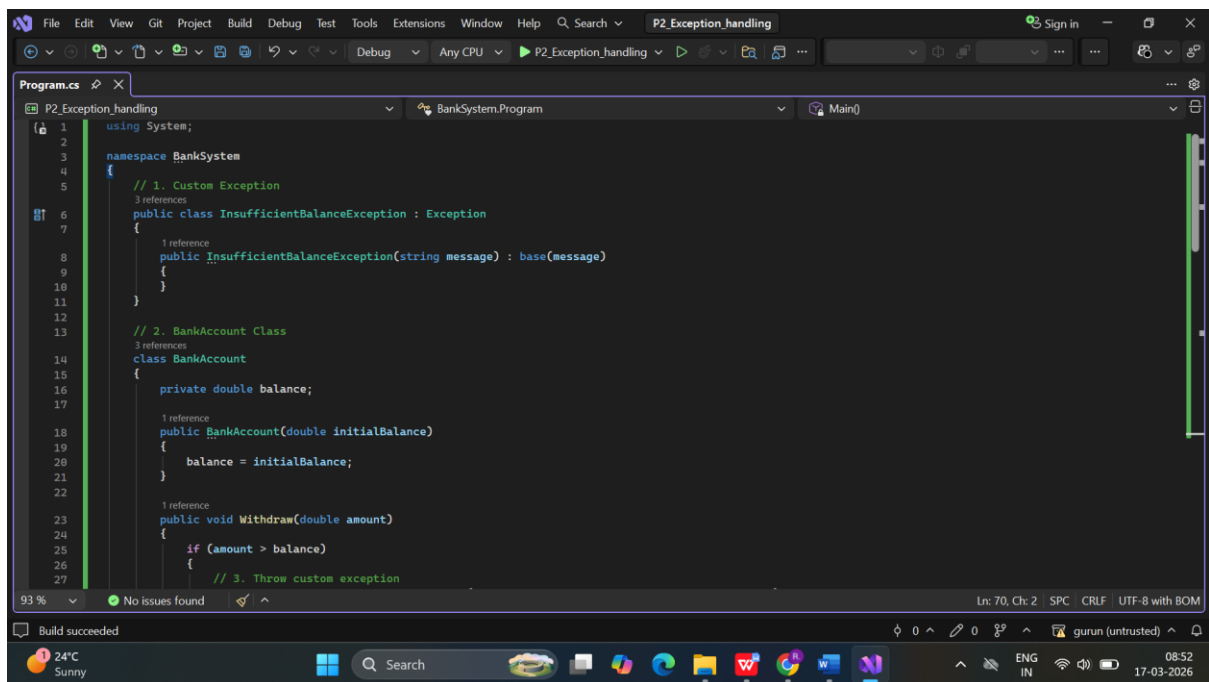
Balance = 3000

Withdraw = 5000

Sample Output:

Error: Withdrawal amount exceeds available balance

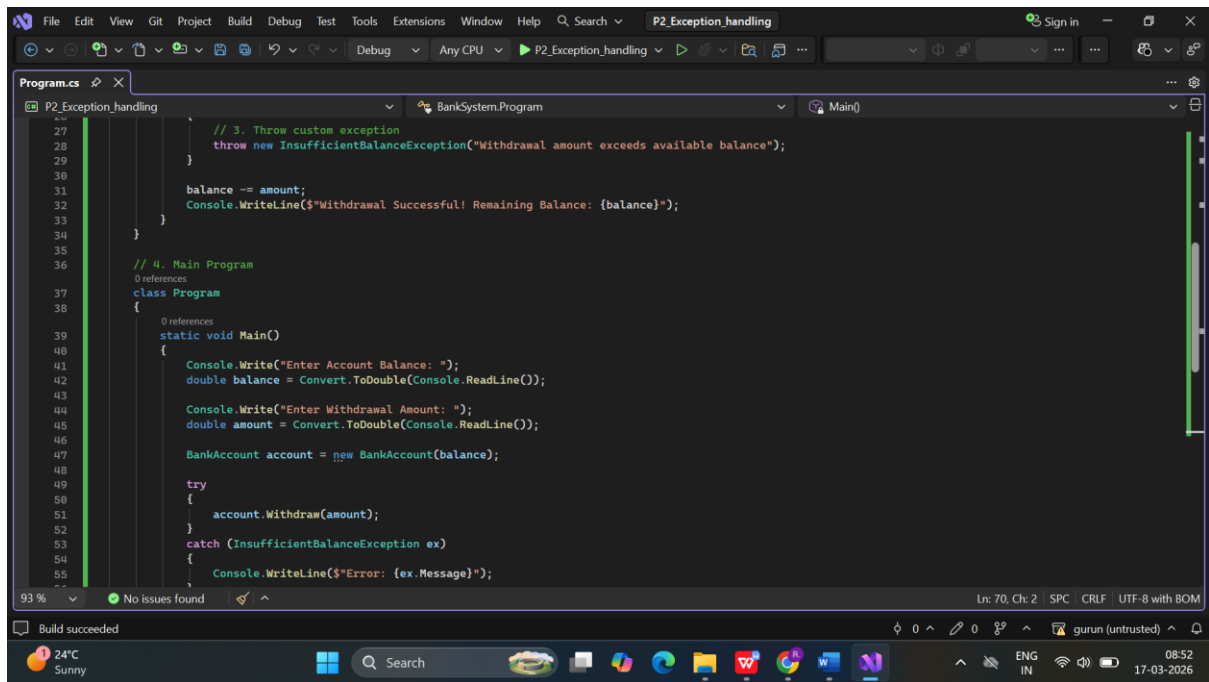
Code Screenshot:



The screenshot shows a Visual Studio IDE window titled "P2_Exception_handling". The code is in C# and defines a custom exception and a bank account class. The code is as follows:

```
1 using System;
2
3 namespace BankSystem
4 {
5     // 1. Custom Exception
6     public class InsufficientBalanceException : Exception
7     {
8         public InsufficientBalanceException(string message) : base(message)
9         {
10         }
11     }
12
13     // 2. BankAccount Class
14     class BankAccount
15     {
16         private double balance;
17
18         public BankAccount(double initialBalance)
19         {
20             balance = initialBalance;
21         }
22
23         public void Withdraw(double amount)
24         {
25             if (amount > balance)
26             {
27                 // 3. Throw custom exception
```

The status bar at the bottom indicates "Build succeeded" and "No issues found". The taskbar at the very bottom shows the system clock as 08:52 on 17-03-2026.

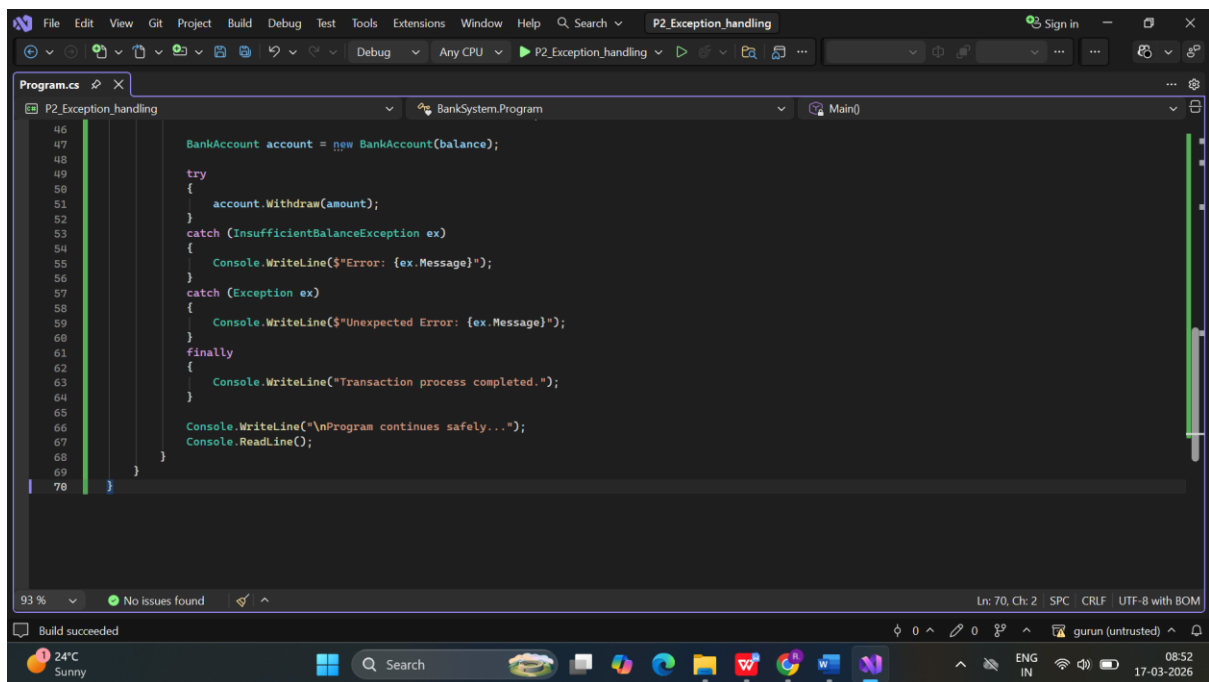


```
27 // 3. Throw custom exception
28 throw new InsufficientBalanceException("Withdrawal amount exceeds available balance");
29
30
31 balance -= amount;
32 Console.WriteLine($"Withdrawal Successful! Remaining Balance: {balance}");
33 }
34 }
35
36 // 4. Main Program
37 class Program
38 {
39     0 references
40     static void Main()
41     {
42         Console.Write("Enter Account Balance: ");
43         double balance = Convert.ToDouble(Console.ReadLine());
44
45         Console.Write("Enter Withdrawal Amount: ");
46         double amount = Convert.ToDouble(Console.ReadLine());
47
48         BankAccount account = new BankAccount(balance);
49
50         try
51         {
52             account.Withdraw(amount);
53         }
54         catch (InsufficientBalanceException ex)
55         {
56             Console.WriteLine($"Error: {ex.Message}");
57         }
58     }
59 }
```

93 % No issues found Ln: 70, Ch: 2 SPC CRLF UTF-8 with BOM

Build succeeded

24°C Sunny 08:52 17-03-2026



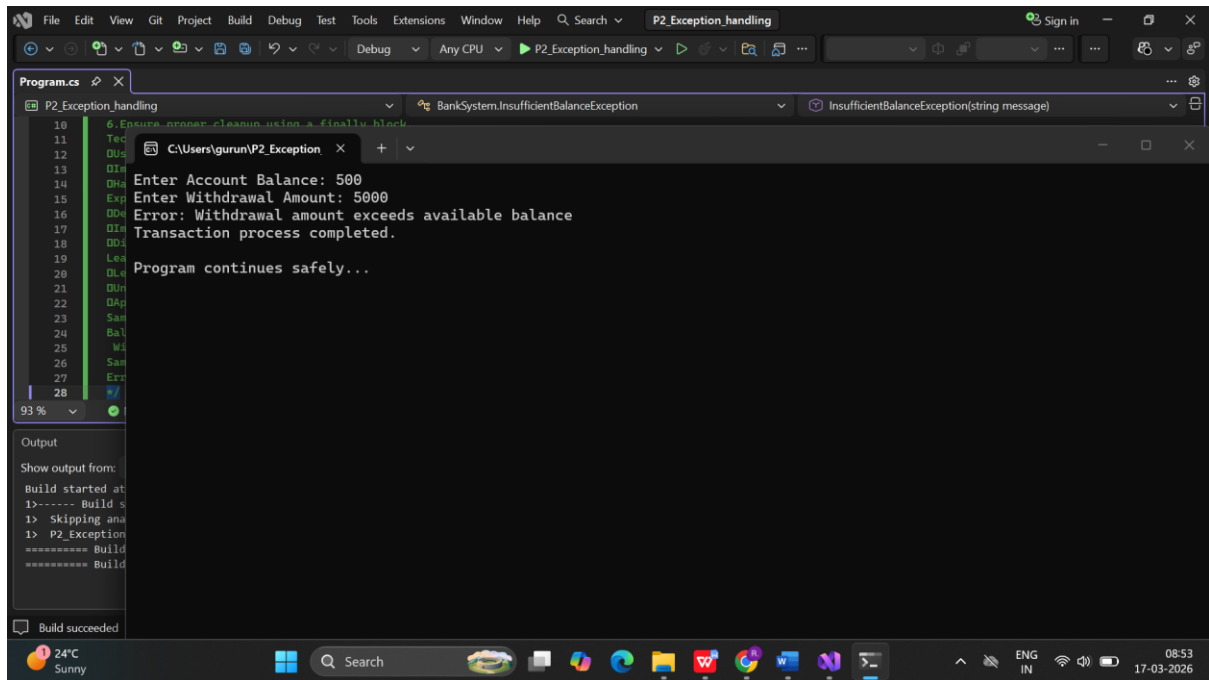
```
46 BankAccount account = new BankAccount(balance);
47
48 try
49 {
50     account.Withdraw(amount);
51 }
52 catch (InsufficientBalanceException ex)
53 {
54     Console.WriteLine($"Error: {ex.Message}");
55 }
56 catch (Exception ex)
57 {
58     Console.WriteLine($"Unexpected Error: {ex.Message}");
59 }
60 finally
61 {
62     Console.WriteLine("Transaction process completed.");
63 }
64
65 Console.WriteLine("\nProgram continues safely...");
66 Console.ReadLine();
67 }
68 }
69 }
```

93 % No issues found Ln: 70, Ch: 2 SPC CRLF UTF-8 with BOM

Build succeeded

24°C Sunny 08:52 17-03-2026

Output Screenshot:



```
10 6. Ensure proper cleanup using a finally block.
11
12
13
14 Enter Account Balance: 500
15 Enter Withdrawal Amount: 5000
16 Error: Withdrawal amount exceeds available balance
17 Transaction process completed.
18
19 Program continues safely...
20
21
22
23
24
25
26
27
28
```

Output

Show output from:

Build started at 1>----- Build s

1> Skipping ana

1> P2_Exception

===== Build

===== Build

Build succeeded

24°C Sunny

Search

ENG IN

08:53 17-03-2026